PY102: Foundations of Data Structures – Week 1 Master Reference Note

1. Introduction to Computational Thinking and Course Structure

The transition from a beginner to a professional programmer is marked by the shift from simply "writing code" to "designing smart solutions." In PY102, the focus moves beyond basic syntax toward computational thinking: understanding how data is stored, how solution steps are organized, and how to evaluate the efficiency of a solution as problems scale. Week 1 establishes the fundamental "how" of data organization, providing the architectural base for all complex software systems.

The Four Units of Mastery

The course is structured into four progressive modules designed to build systems-thinking:

1. **Foundations of Data Structures:** Exploring how computers store and organize information using lists, arrays, and multi-dimensional grids.
2. **Stacks and Queues:** Understanding ordered data handling through "Last-In-First-Out" (LIFO) and "First-In-First-Out" (FIFO) behaviors.
3. **Algorithm Analysis:** Introduction to Big O notation and measuring performance as data grows.
4. **Searching and Sorting:** Implementing core tools like linear search, binary search, and various sorting algorithms.

Grading and Review Philosophy

The grading structure prioritizes consistent participation and the ability to reason through code efficiency and logic.| Component | Weight | Focus Areas || ------ | ------ | ------ || **Reviews 1–3** | 30% (10% each) | Predicting outputs, identifying errors, reasoning about efficiency. || **Last Unit Review** | 60% | Cumulative course mastery and debugging patterns. || **Participation** | 10% | Weekly discussion engagement. || **Bonus Points** | Up to 10 pts | Timely submission and passing of all reviews. |

Academic Integrity Policy: External AI tools are strictly prohibited for all review questions. Reviews are designed to measure your personal understanding and thinking.

Core Learning Objectives

- **Data Storage:** Mastering the mechanics of how information is held in memory.
- **Solution Organization:** Learning to structure logic to handle data reliably and predictably.
- **Scalability:** Recognizing how data structure choice impacts performance as datasets grow.

2. Conceptual Foundations: The Workshop Metaphor

To understand data structures, imagine a computer program as a workshop. You do not throw tools and screws in a pile on the floor; instead, you use boxes, drawers, and labeled shelves to find items quickly. In programming, these "containers" are data structures. Choosing the "right

shape" for your data is a strategic decision that ensures your program remains maintainable and efficient.

Comparative Analysis: Lists vs. Dictionaries

Data structures like lists and dictionaries serve different strategic needs. For example, a simple "milk, eggs, bread" inventory requires different structures depending on whether you need a sequence or specific quantities.| Feature | List (Order-based) | Dictionary (Key-Value based) || ------ | ------ | ------ || **Primary Use** | Keeping items in a specific sequence. | Looking up information by a specific name or key. || **Data Mapping** | Single items: "milk", "eggs" | Pairs: {"milk": 2, "eggs": 12} || **Access Logic** | Access via numerical position (index). | Direct access via the key (e.g., "How many eggs?"). || **Performance** | Best for iteration and maintaining order. | Best for immediate value retrieval by key. |

Structural Selection Logic:

- **Use a List** when the order of items is the priority (e.g., a queue or a ranking).
- **Use a Dictionary** when you need to associate data points (like a name to a value) for direct lookup.

3. The Mechanics of Arrays and Indexing

In computer science, a Python list functions as an **array** : an ordered, index-based collection. Professional environments use zero-based indexing, meaning the first element is at position 0.

Primary and Negative Indexing

Python allows for dual navigation of an array, accessing elements from the front or the back.

```
# High Score Board Example
scores = [1200, 980, 870, 430]

# Positive Indexing (Left to Right)
# [0]: 1200, [1]: 980, [2]: 870, [3]: 430

# Negative Indexing (Right to Left)
# [-4]: 1200, [-3]: 980, [-2]: 870, [-1]: 430
```

The Boundary Rule and the IndexError

- **The Boundary Rule:** For any list, valid indices range from 0 to len(list) - 1.
- **The IndexError Trigger:** Attempting to access len(list) or beyond (one past the end) will crash the program.
- **The Append Operation:** .append(value) adds an item to the "tail" of the array, increasing the length and creating a new final index at len(list) - 1.

4. Dynamic List Mutations and Index Shifting

Updating a value is a "static" change, whereas removing or inserting an element "dynamically" reshapes the list. To avoid logic errors, you must track how these operations shift existing indices.

Updates vs. Removals and Insertions

- **Updates (listi = value):** Length stays the same; no elements shift.
- **Removals/Insertions (pop , insert , remove):** Length changes. Elements to the right of the operation must shift left (on removal) or right (on insertion) to accommodate the change.

Mutation Method Deep Dive

- **pop()** : Removes and returns the element at the end of the list.
- **pop(index)** : Removes the element at a specific position. Everything to the right shifts **left** .
- **remove(value)** : Deletes the first occurrence of a specific value. Use this when the content is known but the position is not.
- **insert(index, value)** : Pushes existing elements at and to the right of the target index one step to the **right** .

Code Trace Synthesis: Mutation Puzzles

Puzzle 1: The Playlist Puzzle

1. **Start:** "Intro", "Song A", "Song B", "Song C"
2. append("Song D") $\rightarrow$ "Intro", "Song A", "Song B", "Song C", "Song D"
3. insert(1, "Ad") $\rightarrow$ "Intro", "Ad", "Song A", "Song B", "Song C", "Song D"
4. pop(3) $\rightarrow$ **Removes "Song B"** .
5. **Final:** "Intro", "Ad", "Song A", "Song C", "Song D"**Puzzle 2: Cinema Seating**
6. **Start:** "Alice", "Bob", "Charlie", "Dana"
7. pop(1) $\rightarrow$ **Removes "Bob"** . List: "Alice", "Charlie", "Dana"
8. insert(2, "Eli") $\rightarrow$ **Inserts "Eli" at index 2** . List: "Alice", "Charlie", "Eli", "Dana"
9. pop(3) $\rightarrow$ **Removes "Dana"** .
10. **Final:** "Alice", "Charlie", "Eli"**Puzzle 3: Grocery List**
11. **Start:** "milk", "eggs", "bread", "cheese", "apples"
12. groceries2 = "cereal" $\rightarrow$ "milk", "eggs", "cereal", "cheese", "apples"
13. pop(0) $\rightarrow$ **Removes "milk"** . List: "eggs", "cereal", "cheese", "apples"
14. pop() $\rightarrow$ **Removes "apples"** . List: "eggs", "cereal", "cheese"
15. pop(-2) $\rightarrow$ **Removes "cereal"** .
16. **Final:** "eggs", "cheese"

5. The Accumulator Pattern and Logic Separation

The **Accumulator Pattern** is the professional standard for building new datasets from existing ones without mutating the original source.

The Three-Step Flow

1. **Initialize:** Create an empty result list (the accumulator).
2. **Iterate:** Loop over the original source list.
3. **Conditional Append:** Decide whether to add a filtered or transformed item to the accumulator.

Filter vs. Transform

- **Filter:** Keeping items based on a condition (e.g., extracting only "Hot Days").
- **Transform:** Changing items before adding them (e.g., applying a discount to prices).

Separation of Concerns (Helper Functions)

Clean loop logic separates "assembly" from "logic." Using helper functions makes code readable and reusable.

```
# Helper function (Logic)
def is_hot(temp):
    return temp >= 70

# Accumulator Pattern (Assembly)
temperatures = [55, 72, 49, 90, 66, 59, 80]
hot_days = []

for temp in temperatures:
    if is_hot(temp):
        hot_days.append(temp)

print(hot_days) # [72, 90, 80]
```

6. Multi-Dimensional Arrays: 2D Grid Navigation

A multi-dimensional array is a "list of lists," essential for representing grids like game boards or cinema seating charts.

The Row Mental Model

Navigation follows the **Two-Index Rule** : the first index selects the row (inner list), and the second index selects the specific column (item within that row).

- **Center Targeting:** On a 3x3 board, board1 targets the center.
- **Corner Targeting:** board0 targets the top-right cell.

Visual Index Mapping (3x3 Grid)

,Col 0,Col 1,Col 2
Row 0,0,0 (Top-Mid),0 (Top-Right)
Row 1,1,1 (Center),1
Row 2,2 (Bottom-Left),2 (Bottom-Mid),2

7. Reliability and Safety Habits (Edge Case Protection)

Defensive Programming is the practice of anticipating potential failures. Using guard clauses and safe operations prevents fragile programs from crashing on unexpected inputs.

The Guard Clause Pattern

Before direct index access, verify the list length.

```
if len(scores) >= 3:
    print(scores[2]) # Safe access
else:
    print("Not enough data")
```

Safe Slicing

Slicing (arr:3) is naturally resistant to IndexError. If a list has only 1 item and you request :3, Python returns the 1 item rather than crashing. This is the preferred method for retrieving ranges safely.

The "Safe Get" Function

The following implementation handles boundary checks for both empty lists and out-of-bounds requests:

```
def safe_get(arr, index):
    # Check if index is within the valid zero-based range
    if 0 <= index < len(arr):
        return arr[index]
    else:
        return "Out of range"
```

8. Final Synthesis: Master Checklist

Mastery of data structures involves re-evaluating the state of your data after every operation.

1. **Zero-Based Awareness:** Always count from 0; recognize that -1 is the last element.
2. **Dynamic Shift Tracking:** Manually re-calculate indices whenever using pop(index) or insert().
3. **Method Selection:** Use remove(value) for known entities and pop(index) for known positions.
4. **Accumulator Standard:** Always build new lists via + loop + append to maintain original data integrity.
5. **2D Navigation:** Use the row syntax for grid-based data.
6. **Defensive Access:** Never use direct index access without a len() guard or a slicing pattern.**End of Note.**